

M1.1: Tokenizer 与 Lexer 词法解析

Theory: 源码到 Token 流的增量切片与 Span 物理定位。

Details: rustc_lexer 采用手写的 DFAs 和流式迭代器，将 utf-8 源码切分为包含 TokenKind（标识符、字面量、操作符等）和 Span（代表行列物理范围的 32 位轻量指针）的数据流。Span 用于在后续所有编译阶段进行错误定位与代码追踪。

Trade-off: 为避免物理字符串拷贝，Token 并不存储其字面值，仅存指针和长度。这使得词法分析极快，但要求前端解析阶段必须持有完整的原始源文件缓冲区内存。

M1.4: 类型推论与 HM 算法

Theory: 类型变量声明、合一化（Unification）与类型方程求解。

Details: 基础算法源于 Hindley-Milner 并附带 Rust 的特征约束。类型系统为每个未显式声明类型的变量分配一个临时的“类型占位变量”，扫描表达式并构建一系列等价关系式。在类型检查（Typeck）阶段通过 Unification Table 进行并查集归并并求解方程。

Trade-off: 复杂的泛型嵌套或长链式流式 API 会让合一化矩阵爆炸，导致类型推导延迟飙升。当类型推导失败时，建议显式书写局部变量类型标注。

M2.1: HIR 至 MIR 降级

Theory: 控制流图（CFG）构建、临时变量分配与语句块扁平化。

Details: 编译器将树状结构的 HIR 解构并扁平化为控制流图（MIR）。HIR 中复杂的嵌套控制结构（如 'match' 和 '||' 运算符）在 MIR 中均被解构为只包含赋值（Assignments）和终端跳转（Terminators）的极简基本块（Basic Blocks）。

Trade-off: 降级消除了解释器和代码生成阶段的歧义，但会导致基本块和分支数量成倍增加，需要额外的 MIR 优化器进行死分支合并。

M2.2: MIR 数据流分析

Theory: 定义-使用（Def-Use）链、活跃变量分析与控制流汇合状态求值。

Details: MIR 使用数据流分析引擎来分析生命周期与所有权。它在 CFG 上对每个基本块执行前向（Forward）或后向（Backward）数据流迭代。通过在 Entry 和 Exit 边界维护 Gen/Kill 集合，计算局部变量的生命期与所有权转移状态。

Trade-off: 迭代数据流算法需要遍历控制流图直至状态机收敛（Fixed-point），算法开销与 CFG 的环路复杂度成正比。

M1.2: 宏展开与名称解析

Theory: 宏嵌套展开、名称导入域绑定与符号索引构建。

Details: rustc_resolve 模块采用双遍遍历算法。第一遍扫描解析显式声明的项并展开声明式/过程式宏，第二遍完成名称绑定。它维护了导入依赖图并构建命名空间（Value, Type, Macro）的解析森林。

Trade-off: 嵌套宏和大范围通配符导入（'use a::*'）会引入极大的解析发散，使名称解析成为编译初期的性能瓶颈。建议规范代码，减少大范围通配符导入。

M1.5: Trait 协同性与一致性

Theory: 特化（Specialization）与孤儿规则等重叠规则静态校验。

Details: Trait 引擎使用“孤儿规则”（Orphan Rules）限制：Trait 本身或实现该 Trait 的 Target 类型中，必须至少有一个属于当前 Crate，以防外部污染。在满足重叠检查的前提下，支持依据更加特异的特化覆盖链重定向调用点。

Trade-off: 孤儿规则牺牲了开发灵活性以换取全局一致性，避免两个不同的第三方包同时为第三个包实现同一个 Trait 导致编译器崩溃。

M2.3: NLL 非词法生命周期

Theory: 控制流图上的点集生存期计算与生命周期自适应收缩。

Details: 非词法生命周期（NLL）彻底淘汰了基于大括号作用域的词法生命周期。NLL 通过计算引用的“实际使用点（Use Points）”，在 CFG 的点（Points）集合上构建细粒度区域约束方程，使引用的生命周期收缩到其最后一个使用点。

Trade-off: 精确计算大幅增加了编译器的静态求解压力，尤其是对于大规模的异步函数和生成器，需要维护数万个生存期点集。

M1.3: AST 至 HIR 降级

Theory: 语法糖剥离、闭包捕获分析与高阶脱糖树表示。

Details: rustc_ast 降级为 rustc_hir。在此过程中，'for' 循环被展开为 'loop' 与 'match'，'if let' 被脱糖为 'match'，同时执行闭包的逃逸分析（捕获环境为 By-Value, By-Ref 或 By-Mut），得到极度简化的强类型 HIR 树。

Trade-off: 降低过程会生成大量隐式临时变量，使得 HIR 比 AST 更加臃肿，极大地消耗内存。rustc 通过 Arena 分配器按块分配物理页来加速 HIR 的构建和后续丢弃。

M1.6: 方法解析与 Deref 强转

Theory: 隐式 Deref 链搜寻、候选方法匹配与方法签名绑定。

Details: 当调用 'x.foo()' 时，编译器会根据 'x' 的静态类型构建一个探查候选链：先尝试 'T'，然后 '&T'，再 '&mut T'，随后对 'T' 进行一级 Deref 并重复此循环（如 'Box<T> -> T -> &T'）。直到找到最匹配的 foo 方法签名。

Trade-off: 多层嵌套的 Deref 强转（如 '&&&&T'）会大幅拉长方法解析候选链的宽度，增加方法选择时间，应适度限制强转深度。

M2.4: 借用校验与冲突判定

Theory: 读写锁模型在生命周期内的重叠校验与悬空引用拦截。

Details: 借用检查器（Borrow Checker）在编译期维护一个局部的借用集合。对于任一活跃借用，其对应的生命周期区间内：如果发现一个对原数据的“写”操作与任何“读借用”或“写借用”重叠，或者生命周期超出了原所有者的生存范围，即判定为借用冲突，输出诊断错误。

Trade-off: 该防线在编译期强制推行了单写多读模型，消除了运行时内存屏障，但编写符合借用规则的代码学习曲线极陡。

M2.5: Drop 细化与清理路径

Theory: Drop 标志插入、移动语义边界识别与异常栈清理分支。

Details: 由于所有权可能会因为分支跳转而动态改变（例如 `if cond drop(x);`），编译器无法在编译期静态确定是否释放该变量。因此，`rustc` 在堆栈中为局部变量插入隐式的布尔 Drop 标志（Drop Flags），在运行期动态判定。此外，它为每个可能抛出 Panic 的语句块建立 Cleanup 边。

Trade-off: Drop 标志会略微增加栈空间消耗。优化器会通过静态生存期分析，尽可能消除已被静态确定释放或未分配变量的 Drop 标志。

M3.2: 动态分派与虚表生成

Theory: Trait Object 内部双指针结构、VTable 生成与函数指针偏移跳转。

Details: `&dyn Trait` 在物理上是一个 128 位宽的双指针（Fat Pointer）：一个指针指向具体的实例数据，另一个指针指向由编译器在只读数据段（`.rodata`）生成的 VTable 结构。VTable 中记录了实例的析构函数（Drop glue）、对齐大小以及 Trait 所声明的所有虚函数物理指针。

Trade-off: 动态分派显著减少了生成的二进制体积，但每次调用都要通过虚表指针进行间接跳转，使得 CPU 无法执行分支预测，也彻底阻断了编译器的内联优化。

M3.5: 链接器桥接与 LTO

Theory: 多 Crate LLVM 字节码链接、跨模块优化与目标平台连接器调用。

Details: 在编译末期，`rustc` 会调用平台本地的连接器（如 `ld`、`lld`、`link.exe`）。当启用链接期优化（LTO）时，`rustc` 不生成汇编机器码，而是将 LLVM Bitcode 直接打包，由链接器中的 LLVM 插件执行跨 Crate 的全局死代码消除与跨包内联。

Trade-off: 开启全局 LTO 会极大地拖慢链接阶段，对物理内存容量要求极高。开发期应关闭 LTO 甚至启用动态链接（`prefer-dynamic`）以提升编译吞吐。

M4.1: Safe/Unsafe 静态边界检查

Theory: Unsafe 块作用域划分、原始指针解引用与静态条件防线。

Details: 编译器会显式隔离安全上下文。在 `unsafe` 块中，允许执行解引用裸指针、调用 `unsafe` 函数、访问/修改可变静态变量、实现 `unsafe trait`。借用检查器对 `unsafe` 块内的常规借用依然执行静态借用分析，但裸指针的读写彻底逃逸出借用检查安全防线。

Trade-off: Unsafe 豁免了借用检查器的强力约束，把并发与内存安全性转嫁给开发者，任何失误（如解引用空指针）都会引发未定义行为崩溃。

M2.6: MIR 级编译优化

Theory: 局部常量传播、死代码消除、函数内联与复制消除。

Details: 在将 MIR 交付给 LLVM 之前，`rustc_mir_transform` 执行一套自研的轻量编译优化。由于 MIR 拥有精准的所有权与数据流信息，编译器能安全地执行常量折叠、局部内联与冗余移动消除，大幅精简进入 LLVM 编译后端的指令流。

Trade-off: 与 LLVM 的大而全不同，MIR 优化只做高性价比的操作。复杂的架构级优化仍交由底层的 LLVM 后端来完成。

M3.3: 内存物理布局与 Auto Traits

Theory: Struct 字段重排优化、对齐校验与 Send/Sync 约束性静态分析。

Details: 为优化内存空间，`rustc` 在非 `#[repr(C)]` 的结构体中会自动重排字段物理顺序（如将大对齐字段置前，消除填充字节 Pad）。同时，编译器根据结构体成员是否全部实现 `Send` 和 `Sync`，自动合成该结构体的 `Auto Trait Bounds`。

Trade-off: 字段重排打碎了 C 语言的顺序内存模型。与 C 进行 ABI 互操作时，必须声明 `#[repr(C)]` 或 `#[repr(transparent)]` 以防止布局崩溃。

M3.6: 增量编译与 Query 系统

Theory: 编译过程拆分为原子 Query、缓存索引与增量失效。

Details: `rustc` 核心是一个查询系统（Query System）。每次编译被拆解为成百上千个原子查询（例如 `type_of(DefId)`）。当源文件被修改时，编译器只对受影响的 AST 节点标记为脏，并根据红黑树依赖图自底向上查找依赖节点，仅重新计算并执行受影响的 Query，直接读取其余部分的缓存。

Trade-off: 查询系统引入了复杂的依赖图维护开销。如果项目架构没有良好模块化，微小的修改可能导致依赖图大面积失效，失去增量性能。

M3.1: 泛型静态分派单态化

Theory: 泛型函数根据具体类型参数复制膨胀与符号去重。

Details: 当 Rust 调用 `foo::<i32>()` 和 `foo::<String>()` 时，编译器在单态化（Monomorphization）阶段会为这两个类型各自复制一份 `foo` 的 MIR 并为其生成特定类型的机器码。

Trade-off: 静态分派实现了极致的执行吞吐（无跳转开销，支持内联），但代价是会导致生成的二进制文件发生严重的“代码膨胀（Code Bloat）”，并拉长编译时间。

M3.4: LLVM IR 代码翻译

Theory: MIR 转换至 LLVM IR、类型系统映射与 Intrinsic 处理。

Details: `rustc_codegen_llvm` 将优化后的 MIR 基本块映射为 LLVM 的 SSA 机器码。它利用 LLVM C++ API 将 Rust 类型转换为 LLVM 等效类型（如 `struct` 到 `llvm::StructType`），并将 MIR 语句翻译为 LLVM 的 IR 指令（如 `br`、`phi`、`alloca`）。

Trade-off: 翻译过程是 `rustc` 与 LLVM 两个复杂进程间的频繁交互，数据转换和序列化开销显著。可以通过使用 `Cranelift` 编译器后端（如 `cg_clif`）进行本地测试以缩短开发编译延时。

M3.7: 堆栈回溯与 Panic 运行时

Theory: Landing Pads 异常捕获、Unwind 栈帧析构与 Panic 中断处理。

Details: 当代码触发 `panic!` 且配置为 `unwind` 时，控制流会触发堆栈析构过程。编译器在可能发生 Panic 的位置生成 Landing Pads，通过 C++ ABI 的 `__Unwind_RaiseException` 递归向上回溯调用栈，依次执行局部变量的析构函数（Drop Glue），直到被 `catch_unwind` 捕获。

Trade-off: Unwind 需要生成大量的 Landing Pads 辅助机器码，显著增加了二进制体积。在嵌入式或无异常要求场景下，应配置 `panic = "abort"`，直接一键调用 `core::intrinsic::abort` 终止进程。

M4.2: 未定义行为与 Miri 解释器

Theory: Stacked Borrows 别名模型、未定义行为运行时检测。

Details: 为检测 Unsafe 代码是否安全, rustc 提供了 Miri 工具。

Miri 是一个解释执行 MIR 的虚拟机。它在运行时维护一个“借用堆栈模型”(Stacked Borrows), 跟踪内存分配上所有借用的重叠与读写历史。一旦发现内存读写违反别名原则(如在读借用活跃时修改了裸指针), 立即报错中断。

Trade-off: Miri 是基于精密堆栈模型的纯模拟虚拟机, 执行效率极低(比原生机器码慢数百倍), 仅能用于单元测试和验证 Unsafe 边界, 无法作为生产级运行时。

M4.3: CTFE 常量求值引擎

Theory: 编译期虚拟机执行、常量函数求值与类型擦除常量缓存。

Details: 编译期函数求值(CTFE)由 rustc 内置的解释器 'rustc_const_eval' 执行。它是一个轻量级、完全受控的编译期虚拟机。当执行 'const' 或者是 'const fn' 时, 该虚拟机直接解释运行其对应的 MIR, 在编译期完成复杂的计算、数组长度推导、甚至是常量哈希表的构建。

Trade-off: 常量求值运行在编译器内部, 无法发起任何系统调用(无网络、无文件 IO), 且执行的计算量被编译器通过严格的指令计数上限控制, 以防止恶意死循环锁死编译器。

M4.4: 线程安全 Send/Sync 约束

Theory: 多线程数据共享与所有权转移在编译期的严苛审查。

Details: Send 和 Sync 属于 Auto Traits (自动合成特征)。当且仅当一个类型的所有成员变量都实现了 'Send' (可在线程间转移所有权), 该类型才被自动实现 'Send'; Sync 亦然。如果将包含 'Rc' (非 Send/Sync) 的结构体尝试传入 'std::thread::spawn', 类型系统在编译期会直接拒绝并输出错误。

Trade-off: 该机制彻底消除了数据竞争(Data Races), 但增加了多线程程序设计的约束度, 需要开发者熟练利用 'Arc'、'Mutex' 等并发同步原语。

M4.5: 宏卫生性与 proc-macro 桥接

Theory: 名称捕获防范、进程间通信 proc-macro 编译器桥。

Details: 声明式宏使用“卫生性”(Hygiene) 机制, 使得宏内定义的局部变量不会意外污染或被调用处的局部变量污染。过程宏(Proc Macros)作为一个独立的动态链接库, 在编译期由 rustc 加载, 两者之间通过底层的编译器桥(Compiler Bridge)通过进程间或模块间 ABI 传递 TokenStream。

Trade-off: 过程宏需要独立的 Crate 编译, 且在宿主编译时会产生动态链接和进程间调用的开销, 应尽量控制其复杂度和数量。

M5.1: 诊断引擎与 suggestions 渲染

Theory: 多样式 Span 高亮、编译错误码输出与一键自动修复建议。

Details: rustc_errors 模块是 Rust 极致体验的窗口。它使用 Span 信息, 在发生错误时定位到磁盘源文件具体的行列, 提取上下文源码, 并渲染彩色高亮输出。同时, 编译器提供 'Suggestions' (修改建议), 如果匹配到特定的错误模式, 会生成带有修正建议的 diff。

Trade-off: 精细的格式化高亮和诊断建议需要耗费一定的计算和 IO 提取时间, 但这对开发效能的提升起到了绝对的作用。

M5.2: Query 缓存与增量底座

Theory: 依赖依赖图构建、红黑节点重打标与脏数据失效算法。

Details: 在增量编译下, 每次查询(Query)都会在内存依赖图(Dep-Graph)中自动关联其输入(Inputs)和输出(Outputs)。当文件变动时, 修改的叶子节点标红。依赖图采用自底向上的双色传播标记, 仅将与变动节点有直接或间接路径连接的依赖 Query 节点判定为“脏”并重新执行。

Trade-off: 依赖图会常驻编译器内存, 对于百万行级别的超大项目, Dep-Graph 维护的节点和边会吃掉大量的宿主内存, 需注意分配。

M5.3: Unstable Feature Gate 栅栏

Theory: 未稳定 API 限制、编译器通道控制与 feature 门控编译。

Details: 每一项尚未稳定的 Rust 语言新特性或底层编译器 API 都被绑定在一个 'feature' 门控标识上。在代码中, 必须声明 '#![feature(xxx)]' 才能启用。rustc 会在解析 AST 阶段强制拦截这部分门控代码, 如果发现当前编译器运行在 'stable' 通道而非 'nightly' 开发通道, 直接终止编译。

Trade-off: 这一门控机制保证了 stable 通道代码的前向兼容性, 使设计不成熟的 API 得以在 nightly 下反复迭代而不破坏线上生产代码。

M5.4: Crate 元数据序列化

Theory: Crate 导出元数据编码(rmeta)、依赖包解析与跨包加载。

Details: 当一个 Crate 编译完成时, rustc_metadata 会将其所有的公共符号、类型签名、Trait 实现、宏定义、甚至 'inline' 函数的 MIR 结构, 全部以极致压缩的二进制格式序列化并写入 'rmeta' (或 'rlib' 包的元数据段) 文件中。当外部引用此 Crate 时, 只需读取并加载该元数据文件。

Trade-off: 序列化元数据对于支持跨 Crate 的泛型单态化和借用检查至关重要, 但元数据读取和解码过程仍然需要消耗显著的 IO 和解析开销。

Zone T1: rustc Core APIs

```
rustc_driver::run_compiler(),

rustc_interface::interface::run_compiler(),

rustc_middle::mir::Body,          rustc_borrowck::borrowck(),

rustc_codegen_ssa::traits::CodegenMethods,

rustc_trait_selection::traits::fulfill(), rustc_resolve::Resolver

Zone T2: Compilation & Execution Errors

borrowck_lifetime_conflict,      trait_selection_overflow,

monomorphization_recursion_limit,

const_eval_undefined_behavior,      meta-

data_serialization_mismatch, macro_hygiene_expansion_error
```